# FOUNDATION OF JAVA LAB SHEET – 7

## Practice Questions Multithreading

**1.** *(Easy)* Write a Java program that creates a thread by extending the Thread class and prints your name 5 times with a 200 ms delay between each print.

**2.** *(Easy)* Rewrite Question 1 using the Runnable interface instead of extending Thread.

**3.** *(Easy)* Write a program that prints the name and priority of the currently executing (main) thread, then changes its name and prints it again.

**4.** *(Easy)* Create two threads that each print their own name and a count from 1 to 5. Run the program three times and note whether the output order changes each time.

**5.** *(Moderate)* Write a program with three threads, each named "Reader", "Writer", and "Logger". Each should sleep for a random time (0-500 ms) before printing its name and a message.

**6.** *(Easy)* Write a program that demonstrates the use of isAlive() to check a thread's status before it starts, while it runs, and after it finishes.

**7.** *(Moderate)* Create a thread that performs a long-running task (a loop summing numbers from 1 to 10,000). Use join() in the main method to ensure the main thread prints the "Task finished" message only after the calculation completes.

**8.** *(Easy)* Write a program that creates a thread, sets its priority to Thread.MAX_PRIORITY, and another thread with Thread.MIN_PRIORITY. Print each thread's priority before starting them.

**9.** *(Moderate)* Create five threads, assign them priorities 1 through 10 in steps of roughly 2 (or any values you choose), start all of them, and observe (over multiple runs) whether higher-priority threads tend to finish their print statements earlier.

**10.** *(Moderate)* A shared BankAccount class has a balance field and a withdraw(int amount) method (not synchronized). Create two threads that each try to withdraw money from the same account 1000 times. Run it and show that the final balance is sometimes incorrect due to a race condition.

**11.** *(Moderate)* Fix Question 10 by making withdraw() a synchronized method, and confirm the final balance is now always correct.

**12.** *(Moderate)* Rewrite Question 11 so that instead of synchronizing the whole method, you use a synchronized block that locks only a dedicated lock object, and only around the statements that read/update the balance.

**13.** *(Moderate)* Implement a simple producer-consumer scenario using wait() and notify() where a producer thread adds numbers (1 to 10) one at a time to a shared single-slot buffer, and a consumer thread reads and prints each number as it becomes available.

**14.** *(Moderate)* Write a program that creates a deadlock situation using two shared objects, Lock1 and Lock2, where two threads try to acquire them in opposite order. Run it and observe that the program never terminates (you will need to stop it manually).

**15.** *(Moderate)* Fix your deadlock program from Question 14 by ensuring both threads acquire Lock1 and Lock2 in the same order, and confirm the program now completes normally.